

RAPPORT TECHNIQUE

Gestion des Produits

Architecture MVC1 — Java EE / Servlets / JSP

Formateur	Mohamed CHERRADI – TDIA, ENSA Hoceima, UAE
Technologie principale	Java EE 8 — Servlets + JSP + JSTL
Serveur	Apache Tomcat 9.x
Build	Maven 3.x
Persistence	Mémoire (ArrayList — Singleton)
Date	Avril 2026

1. Présentation générale du projet

Ce projet est une application web Java EE qui implémente la gestion complète de produits (opérations CRUD : Créer, Lire, Mettre à jour, Supprimer). Il est conçu selon le modèle architectural MVC1 (Model–View–Controller version 1), couplé à une organisation en trois couches logiques distinctes.

L'objectif principal est de permettre à un utilisateur d'interagir avec une liste de produits depuis un navigateur web : ajouter un nouveau produit, consulter la liste, modifier les informations d'un produit existant, le supprimer, ou encore le rechercher par son identifiant.

1.1 Objectifs pédagogiques

Ce projet illustre plusieurs concepts fondamentaux du développement web Java EE :

- La séparation des responsabilités à travers le pattern MVC.
- L'utilisation des Servlets comme contrôleurs HTTP.
- Le rendu dynamique des pages via les JSP (JavaServer Pages) et la bibliothèque JSTL.
- Le pattern DAO (Data Access Object) pour l'accès aux données.
- Le pattern Singleton pour partager un état unique entre plusieurs composants.
- La configuration du déploiement via le fichier web.xml et Maven (pom.xml).

1.2 Technologies utilisées

Technologie	Version	Rôle dans le projet
Java	11+	Langage de programmation principal
Servlet API	4.0.1	Gestion des requêtes / réponses HTTP
JSP API	2.3.3	Génération dynamique des pages HTML
JSTL	1.2	Boucles et conditions dans les JSP
Apache Tomcat	9.x	Serveur d'application web (conteneur)
Maven	3.x	Gestion des dépendances et du build

2. Architecture du projet

Le projet suit une architecture à trois couches bien séparées, combinée avec le pattern MVC1. Cette organisation garantit une bonne maintenabilité du code et une claire séparation des responsabilités.

2.1 Structure des couches

Couche	Package Java	Responsabilité
Modèle (Model)	dao	Classe Produit — représente les données
Accès aux données (DAO)	dao	Interface + implémentation mémoire
Métier (Service)	services	Logique applicative, Singleton
Contrôleur (Controller)	web	Servlets HTTP — une par opération
Vue (View)	webapp	JSP + JSTL — rendu HTML final

2.2 Flux d'une requête MVC1

Chaque action de l'utilisateur suit toujours le même chemin à travers les couches :

```

Navigateur (requête HTTP)
  |
  v
[Servlet - Contrôleur]   (package web)
  |
  v
[Service Métier]         (package services)
  |
  v
[DAO - Accès données]    (package dao)
  |
  v
[Modèle - Produit]       (classe dao.Produit)
  |
  v (forward ou redirect)
[JSP - index.jsp]        (rendu HTML)
  |
  v
Navigateur (réponse HTML)
  
```

2.3 Mapping des URL

URL	Servlet	Méthode HTTP	Action
/listProduits	ListProduitServlet	GET	Lister / Chercher
/addProduit	AddProduitServlet	POST	Ajouter
/deleteProduit	DeleteProduitServlet	GET	Supprimer

/editProduit	EditProduitServlet	GET	Charger formulaire
/updateProduit	UpdateProduitServlet	POST	Mettre à jour

3. Parties importantes et critiques du code

Ce chapitre décrit les éléments du code qui jouent un rôle central dans le bon fonctionnement de l'application. Comprendre ces parties est indispensable pour maintenir ou faire évoluer le projet.

3.1 La classe `Produit.java` — Le Modèle

C'est la classe la plus fondamentale du projet. Elle représente un produit avec ses quatre attributs : identifiant, nom, description, et prix. Tout le reste de l'application manipule des objets de ce type.

```
public class Produit {
    private Long idProduit;
    private String nom;
    private String description;
    private Double prix;

    // Constructeur vide requis pour la liaison automatique
    public Produit() {}

    // Constructeur pour l'ajout (sans ID)
    public Produit(String nom, String description, Double prix) { ... }

    // Getters & Setters ...
}
```

Point critique : le constructeur vide est obligatoire. Sans lui, les frameworks Java (et la JSP lors du pre-remplissage) ne peuvent pas instancier la classe dynamiquement.

3.2 L'interface `ProduitDAO.java` — Contrat d'accès aux données

Cette interface définit les cinq opérations que toute implémentation doit obligatoirement fournir. En travaillant avec l'interface plutôt qu'avec une classe concrète, le code devient facilement remplaçable : on peut passer d'une liste mémoire à une base de données sans changer le reste de l'application.

```
public interface ProduitDAO {
    void addProduit(Produit p);
    void deleteProduit(Long id);
    Produit getProduitById(Long id);
    List<Produit> getAllProduits();
    void updateProduit(Produit p);
}
```

3.3 `ProduitImpl.java` — Persistance en mémoire

C'est l'implémentation concrète du DAO. Elle utilise une `ArrayList` Java comme base de données temporaire. Deux points critiques méritent attention :

Génération des identifiants : l'ID est calculé comme la taille de la liste plus un. Cela fonctionne en phase de test, mais peut poser des problèmes si on supprime des produits (les IDs peuvent se dupliquer). C'est une limitation connue à remplacer par un compteur indépendant en production.

```
// Génération naïve de l'ID — à améliorer pour la production
p.setIdProduit(new Long(produits.size() + 1));
```

La méthode `init()` précharge deux produits de test au démarrage de l'application. Elle est appelée depuis le constructeur du Singleton.

3.4 Le pattern Singleton dans `ProduitMetierImpl.java`

C'est l'élément le plus critique de l'architecture. Puisque les données sont stockées en mémoire, il est vital que toutes les Servlets partagent exactement la même liste. C'est le rôle du Singleton.

```
public class ProduitMetierImpl implements ProduitMetier {

    private static ProduitMetierImpl instance; // instance unique
    private ProduitDAO dao;

    // Constructeur privé — personne ne peut faire 'new ProduitMetierImpl()'
    private ProduitMetierImpl() {
        dao = new ProduitImpl();
        ((ProduitImpl) dao).init();
    }

    // Point d'accès unique
    public static ProduitMetierImpl getInstance() {
        if (instance == null) {
            instance = new ProduitMetierImpl();
        }
        return instance;
    }
}
```

Risque important : cette implémentation du Singleton n'est pas thread-safe. Dans un environnement multi-utilisateurs (ce qui est le cas de Tomcat), deux requêtes simultanées pourraient créer deux instances différentes. La solution est d'utiliser le mot-clé `synchronized` ou la double vérification verrouillée (double-checked locking).

3.5 Les Servlets — Contrôleurs HTTP

Chaque opération CRUD est gérée par une Servlet dédiée. Ce choix correspond au MVC1 : une Servlet par action.

Servlet `AddProduitServlet` — méthode `doPost()` : récupère les paramètres du formulaire, crée un objet `Produit`, l'ajoute via la couche métier, puis fait un forward vers la JSP. Le forward (et non un redirect) est intentionnel pour conserver les attributs de requête.

```
// Récupération des paramètres
String nom = req.getParameter("nom");
```

```

Double prix = Double.parseDouble(req.getParameter("prix")); // peut lever
NumberFormatException

// Ajout via la couche métier
metier.addProduit(new Produit(nom, description, prix));

// Forward vers JSP (les attributs req. sont conservés)
req.setAttribute("listeProduits", metier.getAllProduits());
req.getRequestDispatcher("index.jsp").forward(req, resp);

```

Servlet DeleteProduitServlet — méthode doGet() : utilise sendRedirect() après la suppression, contrairement aux autres Servlets qui utilisent forward(). Ce choix est correct ici : on veut recharger proprement la liste sans risquer une double-suppression si l'utilisateur actualise la page.

Servlet EditProduitServlet — méthode doGet() : charge le produit à modifier et l'injecte dans la requête sous l'attribut produitEdit. La JSP détecte cet attribut pour basculer le formulaire en mode 'modification'.

Servlet ListProduitServlet — méthode doGet() : gère deux cas en une seule Servlet — afficher tous les produits (sans paramètre) ou chercher un produit par ID (avec le paramètre idProduit). Elle inclut un try/catch pour les IDs non numériques, ce qui est un bon réflexe défensif.

3.6 La JSP index.jsp — Vue unique et formulaire dynamique

La JSP est la seule vue de l'application. Elle remplit plusieurs rôles à la fois : formulaire d'ajout, formulaire de modification, moteur de recherche, et tableau de liste. Le mécanisme le plus intelligent est le formulaire dynamique :

```

<!-- Le formulaire change d'action selon le contexte -->
<form action="${produitEdit != null ? 'updateProduit' : 'addProduit'}"
method="post">
    <input type="hidden" name="idProduit" value="${produitEdit.idProduit}" />
    <input type="text" name="nom" value="${produitEdit.nom}" />
    ...
    <input type="submit" value="${produitEdit != null ? 'Modifier' : 'Ajouter'}" />
</form>

```

Si l'attribut produitEdit est présent dans la requête (injecté par EditProduitServlet), le formulaire pointe vers UpdateProduitServlet et les champs sont pré-remplis. Sinon, il agit comme un formulaire d'ajout simple. C'est un usage efficace de l'Expression Language (EL) de JSP.

3.7 Le fichier web.xml — Configuration des Servlets

Ce fichier est le point d'entrée de la configuration de l'application. Il déclare chaque Servlet et l'associe à une URL. Sans ce mapping, Tomcat ne sait pas quelle Servlet appeler pour une URL donnée.

```

<servlet>
    <servlet-name>list</servlet-name>
    <servlet-class>web.ListProduitServlet</servlet-class>

```

```
</servlet>  
<servlet-mapping>  
  <servlet-name>list</servlet-name>  
  <url-pattern>/listProduits</url-pattern>  
</servlet-mapping>
```

Il définit aussi la page de bienvenue (welcome-file) : quand l'utilisateur accède à la racine de l'application, il est redirigé directement vers listProduits, ce qui lance l'affichage de la liste des produits.

4. Étapes d'implémentation du projet

Cette section décrit dans quel ordre le projet a été (ou devrait être) construit, en partant des fondations vers la couche visible.

Étape 1 — Création du projet Maven

Tout commence par la mise en place de la structure Maven et le fichier pom.xml. C'est la fondation qui définit le projet, sa version Java, et ses dépendances externes.

1. Créer un nouveau projet Maven Web App dans Eclipse ou IntelliJ.
2. Configurer le pom.xml avec les dépendances : Servlet API (scope provided), JSP API (scope provided), et JSTL 1.2.
3. Vérifier la structure : src/main/java pour le code Java, src/main/webapp pour les fichiers web.

```
<!-- Dépendances clés dans pom.xml -->
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>jstl</artifactId>
  <version>1.2</version>
</dependency>
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>javax.servlet-api</artifactId>
  <version>4.0.1</version>
  <scope>provided</scope> <!-- Fourni par Tomcat -->
</dependency>
```

Étape 2 — Création du Modèle (classe Produit)

Avant d'écrire quoi que ce soit d'autre, on définit le modèle de données. La classe Produit.java est créée dans le package dao avec ses quatre attributs et tous ses getters/setters, plus les deux constructeurs (vide et avec paramètres).

Étape 3 — Couche DAO

On crée ensuite l'interface ProduitDAO.java qui définit les cinq opérations CRUD sans aucun détail d'implémentation. Puis on écrit ProduitImpl.java qui implémente cette interface avec une ArrayList en mémoire, et la méthode init() qui précharge des données de test.

Étape 4 — Couche Service (Métier)

On crée l'interface ProduitMetier.java qui reprend les mêmes méthodes que ProduitDAO. Puis on implémente ProduitMetierImpl.java en appliquant le pattern Singleton. Cette classe instancie le DAO dans son constructeur privé et expose toutes les opérations.

C'est à cette étape qu'on teste manuellement (avec un main ou en déboguant) que les opérations fonctionnent correctement avant d'ajouter la couche web.

Étape 5 — Configuration du web.xml

Avant d'écrire les Servlets, on prépare le fichier WEB-INF/web.xml. On y déclare le nom de l'application, les fichiers de bienvenue, et on réserve les noms des Servlets et leurs URL (même si elles ne sont pas encore codées).

Étape 6 — Écriture des Servlets (Contrôleurs)

On crée les Servlets dans cet ordre logique :

4. ListProduitServlet — la plus simple, elle récupère la liste et fait un forward vers la JSP.
5. AddProduitServlet — reçoit un formulaire POST, crée un produit, et redirige.
6. DeleteProduitServlet — reçoit un ID en GET, supprime, et redirige.
7. EditProduitServlet — charge un produit et l'injecte dans la requête.
8. UpdateProduitServlet — reçoit les données modifiées en POST et met à jour.

Étape 7 — Création de la JSP (la Vue)

La JSP index.jsp est créée dans webapp/. Elle comprend trois sections principales :

- Le formulaire d'ajout/modification dynamique (avec Expression Language pour détecter produitEdit).
- Le formulaire de recherche par ID (pointant vers listProduits en GET).
- Le tableau JSTL avec la boucle forEach sur listeProduits, et les liens Modifier/Supprimer.

On utilise la balise JSTL `<%@ taglib prefix='c' uri='http://java.sun.com/jsp/jstl/core' %>` pour activer les balises `c:forEach`.

Étape 8 — Test et déploiement

9. Compiler le projet avec `mvn clean package` pour générer le fichier `.war`.
10. Déployer le `.war` dans le dossier `webapps/` de Tomcat (ou via l'intégration Eclipse/IntelliJ).
11. Démarrer Tomcat et accéder à `http://localhost:8080/GestionDesProduitsMVC1/listProduits`.
12. Tester chaque fonctionnalité : ajout, affichage, recherche, modification, suppression.

#	Étape	Fichiers concernés
1	Configuration Maven	pom.xml
2	Classe Modèle	dao/Produit.java
3	Couche DAO	dao/ProduitDAO.java, dao/ProduitImpl.java
4	Couche Service	services/ProduitMetier.java, services/ProduitMetierImpl.java
5	Configuration Web	WEB-INF/web.xml

6	Servlets (Contrôleurs)	web/ListProduitServlet.java, AddProduitServlet.java, ...
7	Vue JSP	webapp/index.jsp
8	Test & Déploiement	target/GestionDesProduitsMVC1.war

5. Fonctionnalités et limites du projet

5.1 Fonctionnalités implémentées

Fonctionnalité	Statut
Afficher la liste des produits	Implémentée — ListProduitServlet
Ajouter un nouveau produit	Implémentée — AddProduitServlet
Modifier un produit existant	Implémentée — EditProduitServlet + UpdateProduitServlet
Supprimer un produit	Implémentée — DeleteProduitServlet
Rechercher par identifiant	Implémentée — ListProduitServlet (paramètre idProduit)
Formulaire pré-rempli à la modification	Implémentée — attribut produitEdit dans JSP
Confirmation avant suppression	Implémentée — alert JavaScript dans index.jsp

5.2 Limites connues et axes d'amélioration

- Persistance volatile : toutes les données sont perdues au redémarrage du serveur. Solution : remplacer ProduitImpl par une implémentation JDBC ou JPA/Hibernate.
- Singleton non thread-safe : l'implémentation actuelle peut créer des doublons en cas de requêtes simultanées. Solution : ajouter synchronized ou utiliser l'initialisation paresseuse verrouillée.
- Génération d'ID fragile : l'ID est calculé par size+1, ce qui peut créer des doublons après des suppressions. Solution : utiliser un compteur AtomicLong ou un auto-increment de base de données.
- Pas de validation des entrées côté serveur : une saisie invalide (prix non numérique) lèvera une exception non gérée. Solution : ajouter des try/catch et retourner des messages d'erreur à l'utilisateur.
- Vue unique (index.jsp) : regrouper toute l'interface dans une seule JSP devient difficile à maintenir si le projet grandit. Solution : séparer en plusieurs JSP ou utiliser des fragments.

5.3 Évolutions possibles

- Remplacer la couche DAO mémoire par une couche JDBC (MySQL / PostgreSQL).
- Ajouter une gestion des utilisateurs et une authentification (Servlet de login, sessions).
- Migrer vers Spring MVC pour un contrôleur centralisé (DispatcherServlet) à la place du MVC1.
- Ajouter des tests unitaires sur la couche service avec JUnit.
- Utiliser une pagination pour afficher les produits par lots.

Conclusion

Ce projet constitue une base solide pour comprendre le développement web en Java EE. Il illustre clairement comment organiser une application en couches indépendantes — DAO, Service, et Contrôleur — et comment les faire communiquer au travers de l'architecture MVC1.

Les concepts mis en oeuvre — Servlets, JSP, JSTL, pattern DAO, Singleton, Maven, et déploiement Tomcat — représentent les fondamentaux que tout développeur Java web doit maîtriser avant d'aborder des frameworks plus avancés comme Spring Boot.

La principale limite du projet, la persistance en mémoire, est volontaire dans un contexte pédagogique : elle permet de se concentrer sur l'architecture web sans complexité supplémentaire. L'étape naturelle suivante serait d'introduire une base de données via JDBC ou JPA pour rendre l'application pleinement opérationnelle.